Actividad: paso de mensajes con UDP y TCP

Objetivo

Analizar los elementos del paso de mensajes e implementar la comunicación entre procesos locales mediante sockets UDP y TCP en Java.

Parte 1. Análisis del paso de mensajes

Para cada implementación, identifiquen:

- Emisor y receptor.
- Mensaje transmitido.
- Canal de comunicación.
- Operaciones de envío y recepción.
- Tipo de direccionamiento.
- Operaciones bloqueantes y no bloqueantes.
- Tipo de comunicación: síncrona, asíncrona o condicional.

Parte 2. Comunicación local con UDP

Implementen dos programas independientes:

Servidor UDP

- Abre un DatagramSocket en el puerto 5000.
- Espera un datagrama mediante receive().
- Obtiene el mensaje, la dirección y el puerto del cliente.
- Devuelve una confirmación: Mensaje recibido: [mensaje].
- Continúa atendiendo mensajes hasta recibir SALIR.

Cliente UDP

- Se comunica con localhost en el puerto 5000.
- Lee un mensaje desde el teclado.
- Lo convierte en bytes y crea un DatagramPacket.
- Envía el mensaje y espera la respuesta.
- Muestra la respuesta recibida.

Ejecuten primero el servidor y después, en otra terminal, el cliente.

Parte 3. Comunicación local con TCP

Implementen un **directorio de servicios** con dos programas independientes.

Servidor TCP

- Abre un ServerSocket en el puerto 6000.
- Espera conexiones mediante accept().
- Guarda servicios en una estructura como HashMap.
- Procesa las siguientes solicitudes:

REGISTRAR nombre puerto

BUSCAR nombre

SALIR

- Responde si el servicio se registró, si fue encontrado o si no existe.
- Atiende solicitudes hasta recibir SALIR.

Cliente TCP

- Se conecta a localhost en el puerto 6000.
- Permite escribir y enviar solicitudes.
- Espera y muestra la respuesta del servidor.
- Cierra el socket al enviar SALIR.

Ejemplo de ejecución:

REGISTRAR calculadora 7000
Servicio registrado

BUSCAR calculadora
Servicio encontrado en el puerto 7000

BUSCAR inventario
Servicio no encontrado

Parte 4. Pruebas y comparación

Realicen al menos tres ejecuciones de cada implementación y registren:

Prueba	Protocolo	Mensaje enviado	Respuesta	Resultado
1	UDP			
2	UDP			
3	UDP			
4	TCP			
5	TCP			
6	TCP			

Respondan brevemente:

1. ¿Qué métodos bloquean la ejecución mientras esperan datos o conexiones?
2. ¿Por qué UDP no necesita accept()?
3. ¿Qué diferencias observaron entre DatagramSocket, Socket y ServerSocket?
4. ¿Cuál protocolo utilizarían para el directorio de servicios y por qué?
5. ¿Qué sucede si ejecutan el cliente antes que el servidor?

Entregables

- Código fuente del cliente y servidor UDP.
- Código fuente del cliente y servidor TCP.
- Capturas de las ejecuciones en terminales separadas.
- Tabla de pruebas.
- Respuestas a las preguntas.
- Diagrama sencillo del intercambio de mensajes de cada implementación.